

Reporte de librerías

Proyecto **bitets** (Flutter)

Inventario de dependencias declaradas en `pubspec.yaml`, con su rol real dentro del código (`lib/`) y una breve descripción general.

Resumen ejecutivo

El proyecto **bitets** es una aplicación *Flutter* (Dart ^3.11.5) para la gestión de horarios de exámenes, con un backend REST en `https://saets.nullpointer.us.kg/api/v1` y soporte *offline-first*. Declara **20 dependencias de producción** y **7 de desarrollo**.

Dependencias de producción: 20 paquetes

Dependencias de desarrollo: 7 paquetes

Generadores de código: build runner

Tamaño de `lib/`: 159 archivos Dart

Tests: 0 (todavía no existe `test/`)

Capa / Feature	Paquetes clave	Responsabilidad
Red	<code>dio</code> , <code>flutter_secure_storage</code>	Cliente HTTP único + tokens
Estado	<code>flutter_riverpod</code> , <code>riverpod_annotation</code>	Gestión de estado global
Persistencia	<code>drift</code> , <code>drift_flutter</code>	SQLite reactivo + caché offline
Routing	<code>go_router</code>	Navegación declarativa
Autenticación	<code>local_auth</code> , <code>device_info_plus</code>	Biometría + device info
Notificaciones	<code>flutter_local_notifications</code> , <code>timezone</code> , <code>flutter_timezone</code>	Recordatorios
Conectividad	<code>connectivity_plus</code>	Detección online/offline
Archivos	<code>path_provider</code> , <code>open_filex</code> , <code>share_plus</code>	Exportar PDF/iCal
UI / Charts	<code>fl_chart</code>	Gráficos del panel admin
Build / lint	<code>build_runner</code> , <code>freezed</code> , <code>json_serializable</code> , <code>drift_dev</code> , <code>riverpod_generator</code> , <code>flutter_lints</code> , <code>flutter_launcher_icons</code>	Generación de código + calidad

Dependencias de producción

Gestión de estado

flutter_riverpod (^3.1.0)

estado 78 imports toda la app

Descripción: Binding de Flutter para Riverpod, la librería de gestión de estado reactivo. Provee `ConsumerWidget`, `ProviderScope`, `WidgetRef` y las APIs `ref.watch` / `ref.read` / `ref.listen`.

Uso en el proyecto: Es la columna vertebral del estado.

- `lib/main.dart:4,20,28,32-46,51` construye el `ProviderContainer`, envuelve la app con `UncontrolledProviderScope` y dispara `authProvider.checkAuthStatus()` en un post-frame.
- `lib/core/router/app_router.dart` lo usa para alimentar el `refreshListenable` del router.
- `lib/features/auth/presentation/providers/auth_provider.dart` es el notifier principal; las páginas `LoginPage`, `UnlockPage`, `HomePage`, `ProfilePage` lo observan.
- `lib/features/examen/presentation/providers/examen_providers.dart` declara 7 notifiers (`ExámenesGrid`, `AlumnoExámenesGrid`, filtros, etc.).
- Todas las features de CRUD (`areas`, `carrera`, `edificio`, `salon`, etc.) usan `GridFormState<T> extends ConsumerState<...>` (`grid_form_state.dart:9-10`).

riverpod_annotation (^4.0.0)

anotaciones 38 '@riverpod' generado

Descripción: Anotaciones consumidas por `riverpod_generator`. Permite declarar providers como clases o funciones simples con `@riverpod`.

Uso en el proyecto: Cada feature tiene un `*_providers.dart` con anotaciones `@riverpod`. Ejemplos:

- `auth_provider.dart:7,18,20` — `@riverpod class Auth extends _$Auth`.
- `examen_providers.dart` — 7 clases anotadas.
- `mapa_providers.dart`, `charts_providers.dart` — providers funcionales.

Los archivos `*.g.dart` resultantes (*generados, commiteados*) son producidos por `build_runner`.

Networking y almacenamiento seguro

dio (^5.4.0)

HTTP 13 imports core/network

Descripción: Cliente HTTP potente para Dart/Flutter, con interceptores, `BaseOptions`, `CancelToken`, `FormData`, etc. Equivalente a `axios` en JS.

Uso en el proyecto: Es el único cliente HTTP. Todo pasa por un singleton en `lib/core/network/dio_client.dart:1,9,12-30,33-39` con `BaseOptions` apuntando a `ApiConstants.baseUrl` y timeouts de 30s. En `kDebugMode` adjunta un `LogInterceptor`.

- `auth_interceptor.dart:1,6,42-71` — `AuthInterceptor extends Interceptor` que inyecta `Authorization: Bearer <token>` y maneja 401.
- `auth_remote_datasource.dart:1,10-37` — `POST /auth/login`, `POST /auth/register`, `GET /auth/me`, `POST /auth/logout`.
- `laravel_grid_datasource.dart:1,7-50` — CRUD genérico usado por **todas** las features de grid (desenrolla el sobre `{ data: ... }` de Laravel).
- `exámenes_sync_service.dart:4,16,48-64` — sincroniza borrados offline.
- `calendar_export.dart:3,32-38,69` — descarga `.pdf` / `.ics` con `ResponseType.bytes`.
- `examen_details_page.dart:1,64-65,73,116-117,125,142-152` — métodos `POST` y `DELETE` sobre la ruta `mis-exámenes` para inscribir y desinscribir al alumno.

flutter_secure_storage (^10.0.0)

keychain 2 imports auth

Descripción: Wrapper multiplataforma para guardar secretos pequeños en los keychains/keystores del sistema (Keychain en iOS, EncryptedSharedPreferences en Android, libsecret en Linux).

Uso en el proyecto:

- `auth_interceptor.dart:2,7-10,28,66` — lee la clave `auth_token` y la borra ante un 401.
- `auth_local_datasource.dart:1,7-10,20,24,29,33,39,43,49,53,60` — gestiona tres claves: `auth_token`, `user_name`, `biometric_enabled`.

Los permisos `USE_FINGERPRINT` / `USE_BIOMETRIC` ya están declarados en `android/app/src/main/AndroidManifest.xml:3-4` y `NSFaceIDUsageDescription` en `ios/Runner/Info.plist:5-6`.

Autenticación

local_auth (^2.3.0)

biometría 1 import features/auth

Descripción: Plugin de Flutter para autenticación biométrica del sistema operativo (Face ID, Touch ID, huella, biometría clase 3).

Uso en el proyecto: `auth_repository_impl.dart:1,14,18,22-23,115-136` usa `LocalAuthentication` para:

- `isDeviceSupported()` — chequeo de capacidad.
- `authenticate(localizedReason: 'Desbloquea bitets para continuar', options: AuthenticationOptions(stickyAuth: true))` — desbloqueo real.

La UI vive en `biometric_setup_dialog.dart` (activación tras primer login) y `unlock_page.dart` (auto-intento en post-frame + botón de fallback).

device_info_plus (^13.0.0)

metadata 1 import features/auth

Descripción: Lee metadatos del dispositivo/SO (modelo, fabricante, SO, versión, etc.) en todas las plataformas.

Uso en el proyecto: `auth_provider.dart:4,44-73` define `_getDeviceName()`, que devuelve un nombre legible del dispositivo y lo envía en el body de `POST /auth/login` y `POST /auth/register` como `deviceName`. Se ramifica por plataforma (`kIsWeb`, `AndroidInfo`, `IosInfo`, `LinuxInfo`, `MacOsInfo`, `WindowsInfo`).

Persistencia local

drift (^2.34.0) + drift_flutter (^0.3.0)

SQLite 4 tablas core/database

Descripción: Drift es un ORM/DSL reactivo para SQLite en Dart. Defines tablas como `class X extends Table`, anotas la base con `@DriftDatabase` y el generador produce una API fuertemente tipada.

Uso en el proyecto:

- `lib/core/database/app_database.dart:1,4,6-71` declara cuatro tablas:
 - `ExamenesCache` — payload JSON de cada `Examen` (campos `pendingDelete`, `pendingDeleteAt` para sincronización).
 - `UserCache` — fila única con el JSON del usuario actual.
 - `NotificacionExamen` — registro de notificaciones programadas para re-programarlas en el boot.
 - `MapaCache` — JSON del canvas del mapa del campus.

- `schemaVersion: 3` con `MigrationStrategy` que crea `notificacionExamen` en v2 y `mapaCache` en v3.
- `AppDatabase.defaults()` usa `drift_flutter` (`driftDatabase(name: 'bitets_cache')`), que elige la implementación SQLite correcta en cada plataforma.
- Consumida por: `exámenes_local_datasource.dart`, `user_local_datasource.dart`, `mapa_local_datasource.dart` y `notifications_service.dart` (vía `_db.notificacionExamen`).
- El archivo generado `app_database.g.dart` (1429 líneas) está commiteado.

Navegación

`go_router` (^14.8.0)

`routing` `1 provider` `core/router`

Descripción: Router declarativo basado en URLs para Flutter. Soporta deep links, navegación con `context.go`, y un `refreshListenable` para redirigir en función del estado.

Uso en el proyecto: `lib/core/router/app_router.dart:3,16,27-58` define `goRouterProvider = Provider<GoRouter>((ref) {...})` con:

- `initialLocation: '/'` → `SplashPage`.
- `refreshListenable: authListenable` (un `ValueNotifier<AuthState>` espejado desde el `auth provider`).
- `redirect` (líneas 31-50): si `loading` → `null`; si `authenticated` → `/admin` o `/home`; si `storedSession` → `/unlock`; si `unauthenticated` → `/login`.
- Rutas: `/`, `/login`, `/unlock`, `/home`, `/admin`.

Serialización JSON

`json_annotation` (^4.12.0)

`anotaciones JSON` `4 modelos`

Descripción: Anotaciones (`@JsonSerializable`, `@JsonKey`, ...) consumidas por `json_serializable`. Empareja con el runtime en `json_serializable` (dev-dep).

Uso en el proyecto: 4 archivos anotados a mano:

- `features/auth/domain/entities/user.dart:1,3,5,12,14,27-29` — `@JsonSerializable()` con `@JsonKey(name: 'created_at'/'updated_at')`.
- `features/auth/data/models/user_model.dart:1,4,6,13,15,28-31` — DTO de `/auth/me` con `toEntity()` / `fromEntity()`.
- `features/auth/data/models/auth_response_model.dart:1,4,7,14-17` — `{ UserModel user; String token; }` de `login/register`.
- `features/grid/data/models/laravel_paginated_response.dart:1,3,5,13-16` — sobre genérico `{ data: [...], links, meta }` de Laravel, usado por **todas** las features grid.

Modelado de estado inmutable

`freezed_annotation` (^3.0.0) + `freezed` (^3.2.3)

`sealed unions` `1 tipo`

Descripción: Anotaciones para `freezed`, que genera data classes inmutables y sealed unions con `copyWith`, `==`, `toString` y pattern matching.

Uso en el proyecto: Un único tipo en `features/auth/presentation/providers/auth_state.dart:1,4,6-17`:

```
@freezed class AuthState with _$AuthState
- _Initial
- _Loading
- _Authenticated(User user)
```

- `_Unauthenticated([String? message])`
- `_StoredSession({String? userName, String? message})`

Esta unión es el corazón de la lógica de redirección del router (`app_router.dart:36-49`). El archivo generado `auth_state.freezed.dart` (462 líneas) está commiteado.

Notificaciones locales

flutter_local_notifications (`^22.0.0`)

`notificaciones` `1 archivo` `core/notificaciones`

Descripción: Plugin multiplataforma para mostrar y programar notificaciones locales (Android, iOS, macOS, Linux, Windows).

Uso en el proyecto: Toda la lógica vive en `lib/core/notifications/notifications_service.dart:6,18-19,40-63,66-99,101-159,161-228`:

- Inicialización por plataforma con `@mipmap/ic_launcher` en Android, `DarwinInitializationSettings` en iOS/macOS, `appUserModelId: 'com.bitets.app'` en Windows.
- `requestPermissions()` — permisos por plataforma.
- `scheduleAt({examenId, fireAt, title, body, tipo})` — programa vía `_plugin.zonedSchedule(...)` con canal Android `bitets_examenes` (`Importance.max`, `Priority.high`), persistiendo en Drift.
- `cancelForExamen(int)` y `cancelById(int)` — cancela y marca como `cancelled: true` en Drift.
- `rescheduleAll()` — re-programa las notificaciones pendientes al iniciar la app (llamado en `main.dart:26`).

Los permisos `POST_NOTIFICATIONS`, `VIBRATE`, `RECEIVE_BOOT_COMPLETED`, `SCHEDULE_EXACT_ALARM`, `USE_EXACT_ALARM` están declarados en `android/app/src/main/AndroidManifest.xml:5-9, 37-45`.

timezone (`^0.11.0`)

`tz database` `1 archivo`

Descripción: Puerto en Rust de la base de datos IANA de zonas horarias. Provee `tz.TZDateTime`, `tz.getLocation`, `tz.setLocalLocation`.

Uso en el proyecto: Solo en `notifications_service.dart:9-10,30,33,113,186`:

- `tz_data.initializeTimeZones();` al inicio.
- `tz.setLocalLocation(tz.getLocation(tzInfo.identifier))` para fijar la zona local.
- `tz.TZDateTime.from(fireAt, tz.local)` antes de pasar la fecha a `zonedSchedule` (requerido para que las notificaciones respeten DST).

flutter_timezone (`^5.1.0`)

`zona local` `1 llamada`

Descripción: Devuelve el identificador IANA de la zona horaria del dispositivo (p. ej. `America/Mexico_City`).

Uso en el proyecto: `notifications_service.dart:8,32` — `final tzInfo = await FlutterTimezone.getLocalTimezone();` para alimentar a `timezone`. Si falla, cae a UTC.

Conectividad

connectivity_plus (`^7.1.1`)

`online/offline` `4 imports`

Descripción: Plugin multiplataforma para conocer el estado de conectividad (`wifi`, `mobile`, `ethernet`, `vpn`, `none`, ...) y observar cambios vía `Stream<List<ConnectivityResult>>`.

Uso en el proyecto:

- `exámenes_sync_service.dart:3,12,17,20-21,31-32` — escucha `Connectivity().onConnectivityChanged` para empujar borrados pendientes cuando vuelve la conexión.
- `alumno_examen_repository.dart:1,197-200` — `_hasConnection()` evita llamadas cuando no hay red.
- `enroll_examen_action.dart:1,44-45` — rechaza inscripciones offline con `SnackBar`.
- `mapa_repository.dart:1,17,40,44-47` — `fetchCanvas()` solo va a la red si hay conexión, si no sirve desde `MapaCache`.

Manejo de archivos

`path_provider` (^2.1.4)

`rutas del SO` `1 uso`

Descripción: Devuelve rutas de directorios propios de la plataforma (`getTemporaryDirectory`, `getApplicationDocumentsDirectory`, etc.).

Uso en el proyecto: Una sola llamada en `calendar_export.dart:7,96` para escribir el PDF/iCal descargado en `getTemporaryDirectory()` antes de abrirlo con `OpenFilex`.

`open_filex` (^4.5.0)

`abrir archivo` `1 uso`

Descripción: Fork de `open_file` que abre una ruta con la app del SO asociada al MIME del archivo (`ACTION_VIEW` en Android, `UIDocumentInteractionController` en iOS, `NSWorkspace` en macOS).

Uso en el proyecto: `calendar_export.dart:6,101,107` — `OpenFilex.open(path, type: mimeType)` dentro de `_openCalendarFile(...)`, camino primario en Android/iOS/macOS. Si devuelve un `ResultType` distinto de `done`, cae al share-sheet.

`share_plus` (^13.1.0)

`share sheet` `1 uso`

Descripción: Abre el panel de compartir del sistema (`Intent.ACTION_SEND` en Android, `UIActivityViewController` en iOS, Web Share API en web).

Uso en el proyecto: `calendar_export.dart:8,59-62` — `SharePlus.instance.share(ShareParams(files: [XFile.fromData(...)], subject: label, fileNameOverrides: [filename]))`, como fallback en Windows, Linux y web.

El `AndroidManifest.xml:71-73` declara `<queries>` con `ACTION_SEND` y `*/*` para que el intent sea resoluble en Android 11+.

Gráficos

`fl_chart` (^1.0.0)

`charts` `2 widgets` `features/admin`

Descripción: Librería pura de Dart para gráficos (línea, barra, pastel, dispersión, radar) con hooks de tema para Material 3.

Uso en el proyecto: Solo en el panel admin:

- `exámenes_por_carrera_chart.dart:1,21,29-87` — `PieChart` con `PieChartSectionData`, agrupa exámenes activos por carrera.
- `inscritos_por_materia_chart.dart:1,21,29-199` — `BarChart` con `BarChartGroupData` / `BarChartRodData`, top-12 materias por inscritos, con `FlTitlesData` rotado y `FlGridData` para el grid horizontal.

Ambos se renderizan dentro de `AdminDashboardPage:85,87` y leen de `exámenesPorCarreraProvider` / `inscritosPorMateriaProvider`.

Dependencias de desarrollo

Pa- que- te	Versión	Rol en el re- po
(S) flutter_lints, com», include: package:flutter_lints/flutter.yaml. [El di- rec- to- rio aúrtest/ no exis- te. flutter co-test rre con ce- ro tests. Es- tá en dev- -deps pa- ra que no fa- lle el ma- ni- fest.],	[Preset de lint. Único consumidor: analysis_options.yaml:1, Activo también custom_lint +, riverpod_lint.],	riverpod_g ^4.0.0+1 [Lee las ano- ta- cio- nes y @riverpod emi- te los com.provider mi- tea- dos.],
, json_serializable, ^user_model.g.dart, auth_response_model.g.dart, laravel_paginated_response.g.dart.], [Or- ques- ta , riverpod_generator , json_serializable , freezed , drift_dev Se in-	, ^6.8.0, [Lee @JsonSerializable / @JsonKey y emite user.g.dart, , ^3.2.3 [Lee y @freezed emi- te (46auth_state lí- neas, com- mi-	freezed ^3.2.3 [Lee y @freezed emi- te (46auth_state lí- neas, com- mi-

Pa- que- te	Versión	Rol en el re- po
vo- ca con dart run build_runner build -- delete- conflicting-], outputs	drift_dev: ^2.34.0; use @DriftDatabase y emite app_database.g.dart (1429 líneas, commiteado).], ^0.14.1	tea- do).],
[Ge- ne- ra los ico- nos de pla- ta- for- ma a par- tir de assets/ con logo.webp fon- do adap- ta- ti- vo #D96704 Con- fi- gu- ra- do en], pubspec.yaml:49-59		

Mapa de capas

Qué librería vive en qué capa/feature del proyecto:

Capa / Feature	Librerías
core/network	dio, flutter_secure_storage
core/database	drift, drift_flutter
core/notifications	flutter_local_notifications, timezone, flutter_timezone, drift
core/router	go_router, flutter_riverpod
features/auth	flutter_riverpod, riverpod_annotation, dio, flutter_secure_storage, local_auth, device_info_plus, json_annotation, freezed_annotation
features/examen	dio, connectivity_plus, share_plus, open_filex, path_provider, flutter_riverpod, riverpod_annotation, drift
features/mapa	dio, connectivity_plus, flutter_riverpod, riverpod_annotation, drift
features/admin	fl_chart, dio, flutter_riverpod, riverpod_annotation
features/grid (CRUD compartido)	dio, json_annotation (LaravelPaginatedResponse), flutter_riverpod
features/{areas,carrera,edificio,planes}	flutter_riverpod, riverpod_annotation, dio (vía GridRepositoryImpl)
Build pipeline	build_runner, riverpod_generator, json_serializable, freezed, drift_dev
Lint / calidad	flutter_lints + custom_lint + riverpod_lint
Iconos de plataforma	flutter_launcher_icons

Notas y hallazgos

- **path_provider** se usa **una sola vez** (export de calendario) pese a ser una dependencia habitual.
- **device_info_plus** se usa **únicamente** en **auth** para etiquetar el dispositivo en el login.
- **freezed** se usa para **un solo tipo** (AuthState); una unión sellada de 5 estados que controla toda la redirección del router.
- **fl_chart** vive **solo** en el panel admin (2 widgets); el resto de la app es lista-basada.
- **flutter_local_notifications** + **timezone** + **flutter_timezone** están concentrados en `core/notifications/notifications_service.dart` y existen exclusivamente para programar recordatorios de exámenes y sobrevivir reinicios (`rescheduleAll()` en el boot).
- No hay `test/` en el repo; `flutter_test` está listado pero *sin uso*.
- No existe **override** `--dart-define=API_BASE_URL`. La URL está hardcodeada a producción en `lib/main.dart::_resolveEndpoint()` y `lib/core/constants/api_constants.dart:static String baseUrl`. Para apuntar a un backend local hay que editar `_resolveEndpoint()`.
- Los archivos `*.g.dart` y `*.freezed.dart` están *commiteados*: los revisores esperan verlos en el diff junto al `.dart` fuente.

Comando de verificación recomendado

Antes de cerrar cualquier cambio en dependencias o modelos anotados:

```
dart run build_runner build --delete-conflicting-outputs
flutter analyze
dart format lib/
flutter test
```